

React is a popular library for building user interfaces created by Facebook. Redux on the other hand is a wonderful state management library for JavaScript apps. Put together they offer the ability to build complex React projects.

Table of Contents

Getting Started

Testing your Components

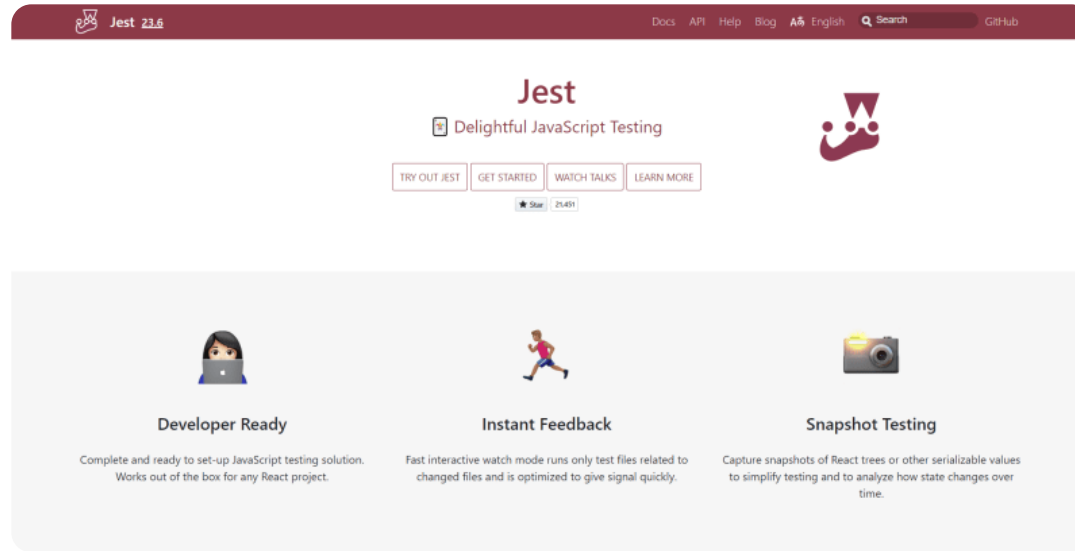
Testing your action creators and async action creators

Testing your reducers

Coverage

Conclusion

Jest is a zero configuration JavaScript testing framework developed by Facebook. It's great for general testing but more so for testing React.



Why is it important to test your applications? Poor user experiences can drive customers away from your applications.

Comprehensive testing allows you to discover errors, inconsistencies, and vulnerabilities in your application.

Practicing test-driven development has been shown to save development costs and time post release. For the purposes of this article, we will simply focus on testing and not test driven development.

Prerequisites

You will need an understanding of how React and Redux applications work. If you are new to this, take a look at this [article](#) about the same.

What We'll Do in this Article

To test JavaScript apps, a good approach to learning is to build and test! It's very hard to see JavaScript testing snippets and make sense of it unless we use it and run tests ourselves.

1. We'll build a to-do app
2. We'll test that to-do app

Getting Started

We will need to install a few (jk its a lot) packages before we can get started.

Get Started w/ JavaScript

for new.

```
yarn add --dev jest babel-jest babel-preset-es2015 babel-pre
```

Don't forget to add a .bablerc file in your project's root folder.

```
// ../.babelrc
```

```
{
```

```
  "presets": ["es2015", "react"]
```

```
}
```

Also add the following to your package.json

```
// ./package.json
"scripts": {
  "test": "jest",
}
```

To run your tests, you will execute the following command

```
yarn test or npm test
```

Our file structure will look like this.

```
- **tests**/  
  --- **snapshots**/ // Folder containing snapshots captured  
  --- actions_test.js.snap  
  --- async_actions_test.js.snap  
  --- reducer_test.js.snap  
  --- todo_component_snapshot_test.js.snap  
  --- actions_test.js // Action creators tests  
  --- async_actions_test.js // Async action creators tests  
  --- reducer_test.js // Reducer tests  
  --- todo_component_test.js // Component tests using enzyme  
  --- todo_component_snapshot_test.js // Component snapshot  
  
- src/  
  --- components/  
  --- Todo.js // Component that displays To-Dos  
  --- redux/
```

```
----- redux/
----- actions.js // app actions
----- reducer.js // app reducer
- node_modules/ // created by npm. holds our dependencies
- .babelrc // contains babel instructions
- package.json
- .gitignore // files and folders to ignore
- README.md // Project description and instructions for running
- yarn.lock // dependency version lock
```

This is a very simplified look at what a React Redux app's folder structure would look like. However it is sufficient for us to grasp the testing concepts. Jest will by default look for test files inside of `__tests__` folder. If you wish to specify your own location, you can pass the `testRegex` option to the Jest configuration object in your `package.json`. For example, if you want to place your test files in a folder named `test_folders`, you would write your Jest configuration as follows.

```
// ./package.json
"jest": {
  "testRegex": "test_folder/.*(js|jsx)$"
}
```

Testing your Components

When you set out to test React components, there are a few primary questions you need to ask yourself.

1. What is the output of the component i.e what does it render?
2. Does the component render different results based on differing conditions?
3. What does the component do with functions passed to it as props?
4. What are the outcomes of a user interacting with the component?

Let's use the following to-do component as an example.

```
// ./src/components/Todo.js

import React, { Component } from 'react'
import PropTypes from 'prop-types'
import moment from 'moment'
import { connect } from 'react-redux'
import { editTodo, toggleTodo, deleteTodo } from '../redux/actions'

export class Todo extends Component {
  constructor() {
    super();
    this.state = {
      formOpen: false,
      todo: {}
    }
  }

  this.handleOpen=this.handleOpen.bind(this);
  this.handleClose=this.handleClose.bind(this);
  this.handleChange=this.handleChange.bind(this);
  this.handleEdit=this.handleEdit.bind(this);
```

```
    this.handleDelete=this.handleDelete.bind(this);  
    this.handleToggle=this.handleToggle.bind(this);  
  }  
  
  // Open Todo edit form  
  handleOpen() {  
    this.setState({formOpen: true});  
  }  
  
  // Close Todo edit form and reset any changes  
  handleClose() {  
    this.setState({formOpen: false});  
    this.setState({todo: {}});  
  }  
  
  // Handle changes to the input fields  
  handleFieldChange(e) {  
    // Property to change e.g title  
    var field = e.target.name;  
    // New value  
    var value = e.target.value;  
    var todo = this.state.todo;
```

```
var todo = Object.assign({}, todo, {[field]: value});  
this.setState({todo: todo});  
}  
  
handleEdit() {  
  // Send to-do id and new details to actions for updating  
  this.props.editTodo(this.props.id, this.state.todo);  
  this.handleClick();  
}  
  
handleDelete() {  
  // Send to-do id to actions for deletion  
  this.props.deleteTodo(this.props.id);  
}  
  
handleToggle() {  
  // Mark a to-do as completed or incomplete  
  this.props.toggleTodo(this.props.id, {done: !this.props.  
}  
  
render() {
```

```
return (  
  <div className="column">  
    <div className="ui brown card">  
      <img className="ui image" src={this.props.url} />  
      {this.state.formOpen ?  
        <div className="content">  
          <div className="ui form">  
            <div className="field">  
              <Label>Title</Label>  
              <input type="text"  
                name="title"  
                defaultValue={this.props.title}  
                onChange={this.handleFieldChange}  
              />  
            </div>  
            <div className="field">  
              <Label>Project</Label>  
              <input type="text"  
                name="project"  
                defaultValue={this.props.project}  
                onChange={this.handleFieldChange}
```

```

    />
  </div>
</div>
</div> :
<div className="content">
  <div className="header">{this.props.title}</div>
  <div className="meta">{this.props.project}</div>
  <div className="meta">Created {moment(this.props.createdAt).format('MM/DD/YYYY')}</div>
</div>
}
<div className="extra_content">
  {this.state.formOpen ?
    <div className="ui two buttons">
      <button className='ui basic green button' onClick={this.handleClick} > Update
      <i className='checkmark icon'></i>
    </button>
      <button className='ui basic red button' onClick={this.handleClick} > Cancel
      <i className='remove icon'></i>
    </button>
    </div> :
    <div>

```

```

    <div className="ui toggle checkbox" style={style}>
      <input type="checkbox" name="public" value={checked} />
      <Label>Complete</Label>
    </div>

    <div className="ui two buttons">
      <button className="ui basic green button">Add</button>
      <button className="ui basic red button">or</button>
    </div>
  </div>
}

</div>
</div>
</div>
)
}
}

Todo.propTypes = {
  id: PropTypes.string.isRequired,
  title: PropTypes.string.isRequired,
  project: PropTypes.string.isRequired,

```

```
done: PropTypes.bool.isRequired,  
url: PropTypes.string.isRequired,  
createdAt: PropTypes.string.isRequired,  
editTodo: PropTypes.func.isRequired,  
toggleTodo: PropTypes.func.isRequired,  
deleteTodo: PropTypes.func.isRequired  
};  
  
export default connect(null, {editTodo, toggleTodo, deleteTodo});
```

You will need to add an actions file at the correct path to prevent a `cannot find module` error from occurring. Add the following to the file. Note that these are not proper actions they are just placeholders to prevent errors when requiring the file in our tests.

```
// ./src/redux/actions.js  
  
export const editTodo = () => {  
  console.log('Sample async function')  
}  
  
export const toggleTodo = () => {  
  console.log('Sample async function')  
}  
  
export const deleteTodo = () => {  
  console.log('Sample async function')  
}
```

You will notice that since we are using Redux, we are using a higher order component(connect()) to inject Redux state in the component. The reason this is important is that when you import this component the following way;

```
import Todo from './Todo'
```

What you get is the wrapper component returned by `connect()`, and not the App component itself. Ideally, you are interested in testing the rendering of the component without the Redux store. If you want to test the connected component, you can wrap it in a `<Provider>` with a store created specifically for this unit test. However, the only reason you would need to do this is to check if your component reacts properly to changes in the store. As React Redux takes care of its own tests, you don't need to do this. You can mock the props as they would appear under different conditions after connecting the component. To assert, manipulate, and traverse your React Components' output we will use Enzyme a JavaScript Testing utility for React by Airbnb. We'll use Enzyme's `shallow` method to render our component. `Shallow` is preferred because it does not render the children of the component whose behavior we do not want to assert. Let's look at what the set up for our tests will look like.

```
// ./**tests*/todo_component_test.js

import React from 'react';
import { shallow } from 'enzyme';
import { Todo } from '../src/components/Todo';
import sinon from 'sinon';

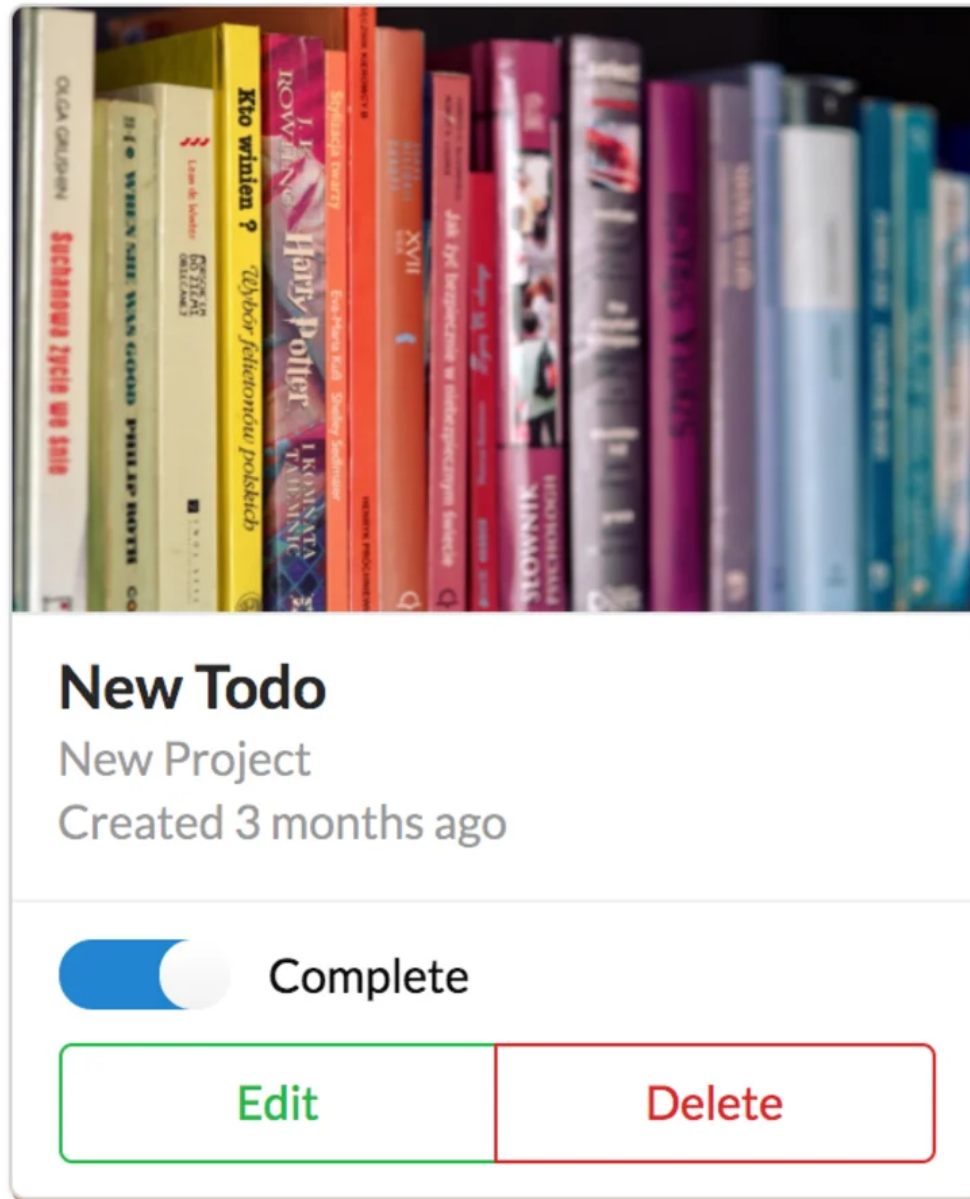
//Use array destructuring to create mock functions.
let [editTodo, toggleTodo, deleteTodo] = new Array(3).fill(

function shallowSetup() {
  // Sample props to pass to our shallow render
  const props = {
    id: "7ae5bfa3-f0d4-4fd3-8a9b-61676d67a3c8",
    title: "Todo",
    project: "Project",
    done: false,
    url: "https://www.photos.com/a_photo",
```

```
      createdAt: "2017-03-02T23:04:38.003Z",
      editTodo: editTodo,
      toggleTodo: toggleTodo,
      deleteTodo: deleteTodo
    }
  }
  // wrapper instance around rendered output
  const enzymeWrapper = shallow(<Todo {...props} />);

  return {
    props,
    enzymeWrapper
  };
}
```

What does this setup do? We define sample props to pass to the component that we are rendering. We have written a function that is available anywhere in our tests that creates an Enzyme wrapper. To answer our first and second questions that we mentioned above, the component renders a card showing the details of a to-do by default and a form to edit the to-do when the edit button is clicked.





Title

Project

☒ **Update** ☐ **Cancel**

We can assert the unique rendered elements of the card as follows:

```
// /**tests*/todo_component_test.js

describe('Shallow rendered Todo Card', () => {
  it('should render a card with the details of the Todo', () {
    // Setup wrapper and assign props.
    const { enzymeWrapper, props } = shallowSetup();

    // enzymeWrapper.find(selector) : Find every node in the
    expect(enzymeWrapper.find('img').hasClass('ui image')).toBe(true);
    expect(enzymeWrapper.find('.header').text()).toBe(props.todo.title);
    expect(enzymeWrapper.find('button.ui.basic.red.button')).toHaveLength(1);
    // enzymeWrapper.containsMatchingElement(node i.e react element)
    expect(enzymeWrapper.containsMatchingElement(<button>Delete)).toBe(true);
  });
});
```

We have used Enzyme's `find` to look for elements that are rendered only when the to-do card is visible. In addition, we can assert against the properties of these elements e.g that they have a certain class or contain certain text. You can take a look at [Enzyme's documentation](#) to see all the matchers you can use to test your component. Go ahead and run `npm test` at the root of your folder to run your tests. Next, we need to assert that the elements that we expect to be present when the form is open are indeed present. For us to do this, we need to simulate a click on the edit button. We can do this as follows

```
const button = wrapper.find('button').first();  
button.simulate('click');
```

We'll need to confirm that the state has changed to reflect that the form is open, input boxes are present and their default values are the original to-do properties and finally, that new buttons with different text are present. This will look like this:

```
// /**tests*/todo_component_test.js

describe('Todo form', () => {
  let wrapper, props_;
  beforeEach(() => {
    // spy on the component handleOpen method
    sinon.spy(Todo.prototype, "handleOpen");
    const { enzymeWrapper, props } = shallowSetup();
    wrapper = enzymeWrapper;
    props_ = props;
  });
  afterEach(() => {
    Todo.prototype.handleOpen.restore();
  });
  it('should update the state property _**`formOpen`**_ and
    // find the edit button and simulate a click on it
    const button = wrapper.find('button').first();
    button.simulate('click');
    // The handleOpen method should be called.
```

```
expect(Todo.prototype.handleOpen.calledOnce).toBe(true);  
// The value of this.state.formOpen should now be true  
expect(wrapper.state().formOpen).toEqual(true);  
});  
  
it('should display different buttons', () => {  
  const button = wrapper.find('button').first();  
  button.simulate('click');  
  // When we click the edit button, the Update button should  
  expect(wrapper.find('button.ui').length).toBe(2);  
  expect(wrapper.find('button.ui.basic.green.button').text()  
});  
  
it('should display current values in edit fields', () => {  
  const button = wrapper.find('button').first();  
  button.simulate('click');  
  // Before any edits are made, the prepopulated values in  
  expect(wrapper.find('input').at(0).props().defaultValue)  
});  
});
```

We are resetting the wrapper before each test using a `beforeEach` to assign a fresh render. You may also notice that we asserted that a component method was called. Using `Sinon`, we can spy on component methods to confirm that they were called and what arguments they were called with. A test spy is a function that records arguments, return value, and exceptions thrown for all its calls.

`sinon.spy(object, "method")` creates a spy that wraps the existing function `object.method`. Here, we are using a spy to wrap an existing method. Unlike spies created as anonymous functions, the wrapped method will behave as normal but we will have access to data about all calls. This helps to ascertain that our component methods are working as expected. An important point to note is that we spy on the component prototype before it's shallow mounted otherwise it will be bound to the original and we'll be unable to assert whether it was called. After each test, we restore the original method so we can have a fresh spy so data from previous tests does not skew results for subsequent tests. Run `npm test` again to ensure your tests are passing. We could have directly manipulated the state to open the form like this:

```
wrapper.setState({ formOpen: true });
```

However, the approach we took allows us to also test user actions. Next, with the form open we can change some values and submit them to see what happens.

```
// ./**tests*/todo_component_test.js

describe('Editing todos', () => {
  let wrapper, props_;

  //In this before each, we are opening the form and changing
  beforeEach(() => {
    // spy on the component handleFieldChange method
    sinon.spy(Todo.prototype, "handleFieldChange");
    // spy on the component handleEdit method
    sinon.spy(Todo.prototype, "handleEdit");
    // spy on the component handleClose method
    sinon.spy(Todo.prototype, "handleClose");
    const { enzymeWrapper, props } = shallowSetup();

    wrapper = enzymeWrapper;
    props_ = props;

    const button = wrapper.find('button').first();
```

```
button.simulate('click');

// find the input field containing the todo title and state
const titleInput = wrapper.find('input').at(0);
titleInput.simulate('change', {
  target: {
    value: 'Changed title',
    name: 'title'
  },
});
});

afterEach(() => {
  Todo.prototype.handleFieldChange.restore();
  Todo.prototype.handleEdit.restore();
  Todo.prototype.handleClose.restore();
});

it('should change state when input values change and call
// this.state.todo should now have a title field with its value
expect(wrapper.state().todo.title).toEqual('Changed title');

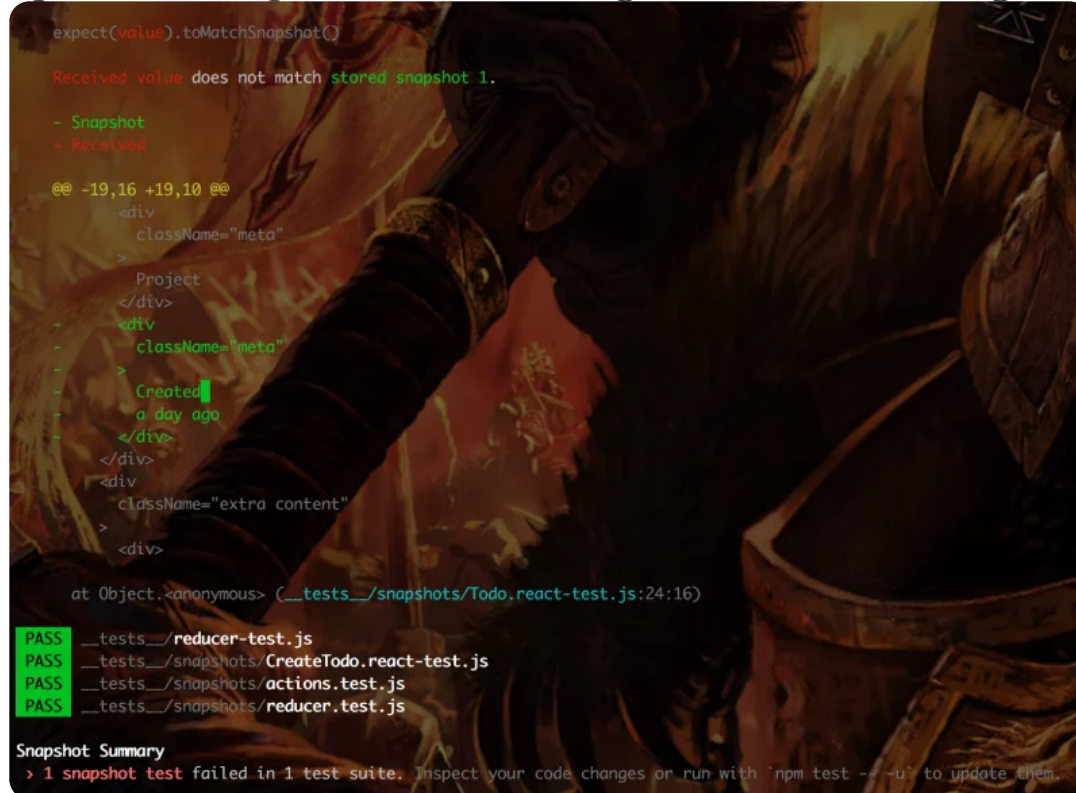
// Since we simulated a change to an input field, the handleFieldChange
expect(Todo.prototype.handleFieldChange.calledOnce).toBe(true);
});
```

```
describe('Submit edits', () => {  
  it('should call handleEdit, editTodo and handleClose when  
    const button = wrapper.find('button.ui.basic.green.button');  
    button.simulate('click');  
  
    // Confirm that the different component methods called  
    expect(Todo.prototype.handleEdit.calledOnce).toBe(true);  
    expect(Todo.prototype.handleClose.calledOnce).toBe(true);  
  
    // the mock function we passed to the renderer instead  
    expect(editTodo).toBeCalledWith(props.id, {"title":  
  });  
});  
});  
});
```

We'll target the different inputs and change the data by simulating changes. Since we are spying on `handleFieldChange` event handler we expect it to be called. If the changes to the inputs are successful, we expect these values to reflect in state so we cross check that too. Finally, when we submit the new values, we expect that the `editTodo` function passed as a prop to the component is called and with the new values. You can use what we have done above to test all the different functions passed as props, component

methods as well as the component itself. Execute the test command again to check on the status of your new tests.

Another way to test components is using snapshots. Snapshots enable you to identify unexpected changes in the UI. The first time you run the test, it creates a reference image which it compares to on subsequent runs. If the new image does not match with the old one, the test fails. Therefore, either an unexpected change occurred or the snapshot needs to be updated. You will be prompted to update the snapshot or make changes and run the test again.



```
expect(value).toMatchSnapshot()

Received value does not match stored snapshot 1.

- Snapshot
+ Received

@@ -19,16 +19,10 @@
  <div
    className="meta"
  >
    Project
  </div>
- <div
-   className="meta"
-   >
-     Created
-     a day ago
-   </div>
</div>
<div
  className="extra content"
>
  <div>

at Object.<anonymous> (__tests__/snapshots/ToDo.react-test.js:24:16)

PASS __tests__/reducer-test.js
PASS __tests__/snapshots/CreateTodo.react-test.js
PASS __tests__/snapshots/actions.test.js
PASS __tests__/snapshots/reducer.test.js

Snapshot Summary
> 1 snapshot test failed in 1 test suite. Inspect your code changes or run with 'npm test -- -u' to update them.
```

You can run Jest with a flag that tells it to regenerate snapshots. However, I prefer to confirm the changes are intentional before updating snapshots. Let's take a look at how we would test our component above using snapshots. First, we'll test the default to-do details view.

```
// ./**tests*/todo_component_snapshot_test.js

import React from 'react';
import toJson from 'enzyme-to-json';
import moment from 'moment';
import { shallow } from 'enzyme';
import { Todo } from '../src/components/Todo';

it('Renders correctly', () => {
  const wrapper = shallow(
    <Todo
      id = '1'
      title = 'Todo'
      project = 'Project'
    />
  );
  expect(toJson(wrapper)).toMatchSnapshot();
});
```

```
done = {false}  
url = "https://www.photos.com/a_photo"  
createdAt = {moment().subtract(1, 'days').format()}  
editTodo = {jest.fn()}  
toggleTodo = {jest.fn()}  
deleteTodo = {jest.fn()}  
/  
);  
expect(toJson(wrapper)).toMatchSnapshot();  
});
```

We render the component and use `enzyme-to-json` to convert the Enzyme wrapper to a format compatible with Jest snapshot testing. You will also notice that we are no longer passing a static `createdAt` prop to the component. This is because we are using `moment's fromNow` function in the component which converts the date supplied to relative time. Therefore, we need to ensure that the time displayed is the same. So we create a `moment` object for the day before the test is run. This means the `createdAt` date in the to-do details view will always be `a day ago`. Run your tests and ensure that the results are as anticipated.

If there are any actions taken by the user that cause the rendered view to look different, you can simulate those actions and take a snapshot of the rendered component.

Testing your action creators and async action creators

Action creators are functions which return plain objects. When testing action creators we assert that the correct action creator was called and the right action was returned. We can test actions on their own but I prefer to test their interaction with the store. We'll use `redux-mock-store`, a mock store for testing your Redux async action creators and middleware. The mock store creates an array of dispatched actions which work as an action log for tests. We can do this as follows

```
// /**tests*/actions_test.js

import configureMockStore from 'redux-mock-store'

// In a real application we would import this from our actions

const createSuccess = (todo) => ({
  type: 'CREATE_SUCCESS',
  todo: todo
})
```

```
type: 'CREATE_SUCCESS',
  todo
});

// Create a mock store
const mockStore = configureMockStore()
const store = mockStore({})
describe('action creators', () => {

  it('creates CREATE_SUCCESS when creating a to-do was successful') {
    // Dispatch the createSuccess action with the values of the
    store.dispatch(createSuccess(
      {
        "id": 1,
        "title": "Example",
        "project": "Testing",
        "createdAt": "2017-03-02T23:04:38.003Z",
        "modifiedAt": "2017-03-22T16:44:29.034Z"
      }
    ));
    expect(store.getActions()).toMatchSnapshot();
  });
});
```

```
});
```

For the above action, calling `store.getActions` will return this:

```
[
  {
    "type": "CREATE_SUCCESS",
    "todo": {
      "id": 1,
      "title": "Example",
      "project": "Testing",
      "createdAt": "2017-03-02T23:04:38.003Z",
      "modifiedAt": "2017-03-22T16:44:29.034Z"
    }
  }
]
```

To avoid having to type this all and having to make changes to expected actions every time we change our code, we can use snapshots instead of typing out the data again. Now, let's look at async action creators. For async action creators using Redux Thunk or other middleware, we'll completely mock the Redux store and use Nock to mock the HTTP requests.

Example

```
// ./**tests**/async_actions_test.js

import configureMockStore from 'redux-mock-store'
import thunk from 'redux-thunk'
import nock from 'nock'
import fetch from 'isomorphic-fetch'

const middlewares = [ thunk ]
```

```
const mockStore = configureMockStore(middlewares)
```

```
// We would import these from actions in real case testing.
```

```
const success = (todos) => ({  
  type: 'FETCH_TODO_SUCCESS',  
  todos  
});
```

```
// Async action creator to fetch to-dos from an API
```

```
const fetchTodos = () => (dispatch) => {  
  return fetch('https://localhost:8000/api/todos')  
    .then(response => {  
      return response.json()  
    })  
    .then(json => {  
      dispatch(success(json));  
    })  
  }  
}
```

```
describe('async actions', () => {  
  let store;
```

```
let fetchTodosData = [
  {
    "id": 1,
    "title": "Example",
    "project": "Testing",
    "createdAt": "2017-03-02T23:04:38.003Z",
    "modifiedAt": "2017-03-22T16:44:29.034Z"
  }
];

beforeEach(() => {
  store = mockStore({});
});

afterEach(() => {
  // clear all HTTP mocks after each test
 nock.cleanAll();
});

it('creates FETCH_TODO_SUCCESS when fetching to-dos has been successful', () => {
  // Simulate a successful response
  nock('https://localhost:8000')
    .get('/api/todos') // Route to catch and mock
```

```
.reply(200, fetchTodosData); // Mock reponse code and  
  
// Dispatch action to fetch to-dos  
return store.dispatch(fetchTodos())  
  .then(() => { // return of async actions  
    expect(store.getActions()).toMatchSnapshot();  
  })  
});  
});
```

When we call the `fetchTodos` async action creator, it makes a request to an API to get to-dos. Once we have received the response it then dispatches the `FETCH_TODO_SUCCESS` action with the array of to-dos received from the API. We have only tested an action that sends out a `GET` request. When you are testing `POST` request, it is very important that the options passed are the same as the request body in your async action creator. I find the easiest way to do this is to stringify the sample request body using `JSON.stringify` then supply it to Nock. Don't forget to run your newly added tests before proceeding.

Testing your reducers

Reducers return a new state after applying actions to the previous state. [Sample reducer](#)

```
// ./src/redux/reducer.js
export const initialState = { // Exporting it for test purposes
  requesting: false,
  todos: [],
  error: null,
};

export default function reducer(state = initialState, action) {
  switch (action.type) {

    case 'TODO_REQUEST':
      // Changes the requesting field in state to true to show loading
      return Object.assign({}, state, {
        requesting: true
      });
  }
}
```

```
    requesting: true,  
    error: null,  
  },  
  
  // State should look like this:  
  // {  
  //   requesting: true,  
  //   todos: [],  
  //   error: null,  
  // }  
  );  
  
  default:  
    return state;  
  }  
}
```

When testing a reducer, you pass it the initial state and then the action created. The output returned should match the new state. Looking at the reducer above, we can see that in the initial state, the requesting field is set to `false`. When an action whose type is `TODO_REQUEST` is dispatched, the field is changed to `true`. In our test, we'll dispatch this action and

confirm that the returned state has changed accordingly. We will be taking the same approach of matching the output to snapshots as we've done before. A test of the reducer defined above would look like this:

```
// ./**tests**/reducer_test.js

import reducer from '../src/redux/reducer'
import {initialState} from '../src/redux/reducer'

describe('todos reducer', () => {
  it('should return the initial state', () => {
    expect(reducer(undefined, {})).toMatchSnapshot()
  })

  it('should handle TODO_REQUEST', () => {
    expect(
      reducer(initialState, {
        type: 'TODO_REQUEST'
```

```
    })  
  ).toMatchSnapshot()  
  })  
})
```

Finally, run `npm test` to ascertain your tests are passing.

Coverage

Jest provides a very simple way to generate coverage. To do this, run:

```
npm test -- --coverage
```

This will produce a coverage folder in your root directory with all the coverage information.

```
> jest "--coverage"
```

```
PASS __tests__/todo-component-test.js
PASS __tests__/actions-test.js
PASS __tests__/reducer-test.js
PASS __tests__/create-todo-component-test.js
PASS __tests__/todolist-component-test.js
PASS __tests__/snapshots/TodoList.react-test.js
PASS __tests__/snapshots/actions.test.js
PASS __tests__/snapshots/Todo.react-test.js
PASS __tests__/snapshots/CreateTodo.react-test.js
PASS __tests__/snapshots/reducer.test.js
```

File	% Stmts	% Branch	% Funcs	% Lines	Uncovered Lines
All files	99.33	88.24	98.44	99.21	
components	97.83	100	94.74	97.83	
CreateTodo.js	100	100	100	100	
Todo.js	100	100	100	100	
Todolist.js	83.33	100	80	83.33	70
redux/modules/todo	100	84.62	100	100	
actions.js	100	75	100	100	
reducer.js	100	94.74	100	100	

```
Test Suites: 10 passed, 10 total
Tests: 77 passed, 77 total
Snapshots: 33 passed, 33 total
Time: 5.902s
Ran all test suites.
```

Conclusion

Jest makes it very easy to test React applications. In addition, by leveraging Enzyme's API, we are able to easily traverse components and test them. We have barely scratched the surface of what these packages have to offer. There is a lot more you can do with them and as a result, you will always be one step ahead of nasty surprises. If you want

to see a complete React Redux running app with fully featured tests, you can take a look [here](#). So go on and try Jest today if you haven't.